Monotone Classification with Relative Approximations

Lecture Notes for SC2320 Data Analytics and Mining

April 11, 2026

Instructions to readers

The flow of this set of notes is in tandem to the flow of the original paper. It will also go in the same flow as the presentation slides. Concepts/Material/Information that is auxiliary to the core ideas of this paper will be outlined at the end of each section.

1 Background and Motivation

1.1 The Challenge of Entity Matching

Definition 1.1 (Entity Matching)**.** Given two sets of entities E_1 and E_2, the task is to determine, for every pair $(e_1, e_2) \in E_1 \times E_2$, whether e_1 and e_2 refer to the same real-world entity. It is a binary classification task that determines whether a candidate pair is a match $(+1)$ or non-match (-1) [2].

To motivate the problem, consider the following scenario where we have two sets of `entities` that we refer to here as product listings - E_1 and E_2, sourced from two different e-commerce platforms — say Shopee and Lazada respectively. Each listing carries attributes such as `p-name`, `p-description`, `p-year`, and `price`. The goal of Entity Matching in this scenario is to decide, for every pair $(e_1, e_2) \in E_1 \times E_2$, if they are referring to the same real-world product.

What makes this non-trivial is that even a genuinely matching pair may disagree on attribute values. For instance, one platform may list a product as *"Sony WH-1000XM5"* while another lists it as *"Sony Noise Cancelling Headphones XM5 Series"*. They are referring to the same product but with different representations. It is therefore insufficient to rely solely on attribute-level comparison to determine a match. To attain maximum accuracy in entity matching, it falls upon a human expert to manually inspect each pair and make a verdict. However, with thousands of listings on each platform, the number of pairs in $E_1 \times E_2$ grows quadratically - making manual inspection labor intensive and expensive. This motivates the need for a more systematic and efficient approach to Entity Matching.

1.2 The Entity Resolution (ER) Pipeline

To address the Entity Matching Problem, we transform it into a classification problem. This is done by passing datasets through an ER Pipeline consisting of three key steps: **Blocking**, **Comparison**, and **Matching**. The first two steps serve as preprocessing stages that progressively reduce and structure the problem, while the third step is what Entity Matching entails and is the primary focus of the paper. We will go over the first two steps in brief before diving into greater detail on the third.

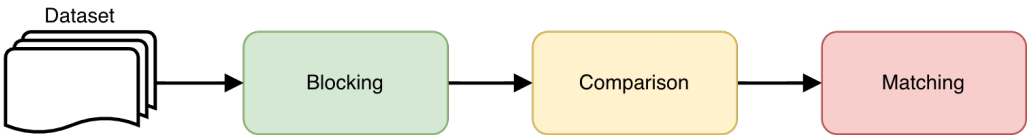

Figure 1: The ER Pipeline.

1.2.1 Blocking

As mentioned earlier in 1.1, the two entity sets E_1 and E_2 can grow quadratically, making it computationally infeasible to compare every pair. The goal of blocking is to narrow the search space down to a smaller candidate subset $S \subseteq E_1 \times E_2$, retaining only pairs that are plausible matches based on an application-dependent heuristic.

Going back to the Shopee-Lazada example, one may restrict S to only those pairs (e_1, e_2) where `p-year` agrees across both listings. Note that blocking is optional — if skipped, $S = E_1 \times E_2$.

Remark 1.1 (Application-Dependent Heuristic)**.** The criterion used for blocking is **application-dependent**; there is no universal rule for what constitutes an "obvious non-match".

For example:

- **E-commerce:** A mismatched `p-year` is a reliable signal to discard a pair.
- **Hospital records:** Mismatched blood type might serve that role.
- **Legal documents:** Neither attribute would reliably apply.

The heuristic is therefore dictated by which attributes are reliable and meaningful in the specific domain.

A notable example of a more general, domain-agnostic blocking heuristic is **Locality Sensitive Hashing (LSH)**. It groups similar pairs into candidate buckets based on hash collisions, this was covered in Chapter 3 of this course.

1.2.2 Comparison

For each remaining pair $(e_1, e_2) \in S$, the comparison step quantifies their similarity across d attributes using a set of similarity functions $sim_1, sim_2, \ldots, sim_d$. The i-th coordinate of the resulting point p_{e_1,e_2} equals $sim_i(e_1, e_2)$, where a higher value indicates greater similarity on that attribute. The choice of similarity function depends on the attribute type:

- For numerical attributes such as `price`, one may use $-|e_1.\texttt{price} - e_2.\texttt{price}|$, where the negation ensures consistency with the convention that higher values reflect greater similarity.
- For text attributes such as `p-name` and `p-description`, functions such as edit distance, Jaccard similarity, or cosine similarity may be employed.

Example 1.1. Consider an entity pair $(e_1, e_2) \in S$. e_1 is a Shopee listing and e_2 is a Lazada listing for what may be the same product. Suppose each entity has the attributes `p-name`, `p-description`, `p-year`, and `p-price`. We pass each attribute pair into a pre-defined similarity function as follows:

- `p-name`: a text attribute — use Jaccard similarity. E.g., *"Sony WH-1000XM5"* vs *"Sony Noise Cancelling XM5"* $\rightarrow sim_1(e_1, e_2) = 0.8$
- `p-description`: a long text attribute — use cosine similarity. $\rightarrow sim_2(e_1, e_2) = 0.65$
- `p-year`: a numerical attribute — use exact match, returning 1 if equal and 0 otherwise. $\rightarrow sim_3(e_1, e_2) = 1$
- `price`: a numerical attribute — use $-|e_1.\texttt{price} - e_2.\texttt{price}|$. E.g., \$350 vs \$345 $\rightarrow sim_4(e_1, e_2) = -5$

The resulting point representing this pair in $\mathbb{R}^4$ is:

$$p_{e_1,e_2} = (0.8,\ 0.65,\ 1,\ -5)$$

This will be done for every pair in S.

This process yields a d-dimensional set of points $P = \{p_{e_1,e_2} \mid (e_1, e_2) \in S\}$ which is handed over to the Matching step.

1.2.3 Matching

The entity matching objective now becomes a binary classification task of assigning each point $p_{e_1,e_2} \in P$ to its correct ground truth label, which is $+1$ for a matching pair and -1 otherwise. While the ultimate approach to this would be with human judgement, this paper introduces **Monotone Classification** as a way to automate the process while ensuring a high degree of labelling accuracy.

1.3 Monotone Classification

Instead of relying on human judgement, we want to find a classifier to correctly label matching and non-matching entities.

The input is the set P of n points in $\mathbb{R}^d$ for $d \geq 1$ that we obtained from the Comparison step. Each point $p \in P$ holds a label from $\{-1, 1\}$ which is represented as $label(p)$.

We then define a classifier as a function:

$$h : \mathbb{R}^d \rightarrow \{-1, +1\}$$

where $h(p) = label(p)$ is a correct classification made by h and will be a misclassification otherwise.

The question is, what would make a good classifier in this context?

1.3.1 The Monotonicity Constraint

To answer this, we first introduce the notion of **dominance**. We say that a point $p \in \mathbb{R}^d$ *dominates* another point $q \in \mathbb{R}^d$, written $p \succ q$, if:

$$p[i] \geq q[i] \quad \text{for all } i \in [d]$$

That is, p scores at least as high as q on every attribute. In the context of entity matching, this means that the pair represented by p is at least as similar as the pair represented by q on every feature.

This supports the **Monotonicity Constraint**. A classifier h is said to be **monotone** if:

$$p \succ q \implies h(p) \geq h(q)$$

Intuitively, if a pair p is at least as similar as q on every attribute, it would be contradictory for h to label p as a non-match (-1) while labeling q as a match $(+1)$. A more similar pair should receive at least as positive a label. We denote the set of all monotone classifiers as $\mathbb{H}_{mon}$. So for entity matching, the key criteria of our classifier is that it must be **monotone**.

1.3.2 The Importance of Monotonicity

Remember that similarity scores are constructed in a way that higher values would indicate greater similarity for each attribute. If a pair p has similarity scores higher than or equal to pair q on all attributes, thereby making p dominate q, it would be in direct contradiction of the aforementioned logic of greater similarity should a classifier label p as a non-match while q as a match. It's like saying: "the less similar pair is a match, the more similar pair is not.". Thus, any classifier that violates the monotonicity constraint should not be used.

1.3.3 The Optimal Monotone Error

Given a classifier h, its **error** on P is defined as the number of points it misclassifies:

$$err_P(h) = \sum_{p \in P} \mathbf{1}_{h(p) \neq label(p)}$$

That is, the total count of points in P whose assigned label differs from their ground truth. A classifier $h \in \mathbb{H}_{mon}$ is said to be **optimal** on P if it achieves the smallest possible error among all monotone classifiers. We define this minimum achievable error as the **Optimal Monotone Error**:

$$k^* = \min_{h \in \mathbb{H}_{mon}} err_P(h)$$

Note that k^* need not be zero — in practice, the labels in P may not perfectly obey monotonicity due to noise in the data, meaning even the best monotone classifier will misclassify some points.

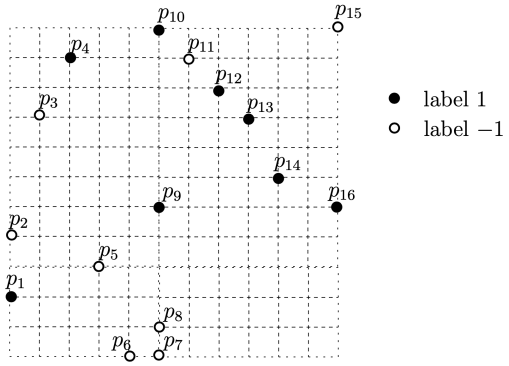

Figure 2: Input set P

Example 1.2. Consider the 2D point set P illustrated in Figure 2. Each point $p_{e_1,e_2} \in P$ represents an entity pair with $d = 2$ similarity features. Black points carry label $+1$ (match) and white points carry label -1 (non-match). A monotone classifier h that maps all black points to $+1$ except p_1, and all white points to -1 except p_{11} and p_{15}, achieves $err_P(h) = 3$. No other monotone classifier does better on P, and hence $k^* = 3$. Note that $k^* > 0$ — this is the non-realizable case, as no monotone classifier can perfectly separate all points in P.

We now want an algorithm $\mathcal{A}$ to find a classifier from $\mathbb{H}_{mon}$ with low error on P. Note that in the beginning, the labels on all points in P are hidden. In order for $\mathcal{A}$ to gain better understanding in forming an optimal classifier, it is allowed to query an *oracle* — a human expert in practice – to reveal the ground truth label of a chosen point $p \in P$. This action is defined as a *probe*. To assess the efficiency of the $\mathcal{A}$, we can measure its cost by the number of *probes* it makes.

To find an optimal monotone classifer on P, one can simply have $\mathcal{A}$ to probe all of P with a cost of n. On the other hand, not probing at all would incur zero cost but the classifier may perform poorly on P. This presents us with the challenge of minimizing the number of probes while still producing a classifier with sufficiently low error.

Formally, given a tolerance parameter $\epsilon \geq 0$, the goal is to determine the minimum number of probes required to guarantee a classifier with error at most $(1+\epsilon) \cdot k^*$. This is defined formally below.

Problem 1.1 (Monotone Classification with Relative Precision Guarantees)**.** Given a real value $\epsilon \geq 0$, find a monotone classifier $h \in \mathbb{H}_{mon}$ whose error on P is at most $(1+\epsilon) \cdot k^*$, while minimizing the number of probes made by $\mathcal{A}$.

Note that when $\epsilon = 0$, the algorithm must find an exactly optimal classifier, which as we will see requires $\Omega(n)$ probes in the worst case — essentially probing all of P. This justifies the introduction of $\epsilon > 0$, where even a small tolerance dramatically reduces the probing cost. The remainder of this paper studies how efficiently this problem can be solved across the full range of ϵ.

1.4 Active Classification and Learning

To motivate Active Classification, we first revisit a familiar example of **passive classification** — the **Perceptron Algorithm**, covered in SC2300 Machine Learning. Recall that the Perceptron is a mistake-driven algorithm that iterates through all training examples $S_n = \{(x^{(t)}, y^{(t)}), t = 1, \ldots, n\}$, where both the points *and* their labels are fully visible from the start. It updates its parameter vector θ whenever a misclassification occurs, converging to a linear classifier that correctly separates all training examples — provided they are linearly separable. This is the **realizable case**.

Remark 1.2 (Realizable Case)**.** A classification problem is **realizable** if there exists a classifier $h \in \mathbb{H}_{mon}$ that correctly classifies all points in P with zero error, i.e., $k^* = 0$. This is analogous to linear separability in the Perceptron setting.

When data is not linearly separable, the Perceptron fails to converge — motivating the use of **hinge loss** to tolerate a margin of error. The same challenge arises here: in practice, the labels in P rarely obey monotonicity perfectly, meaning no monotone classifier can achieve zero error. This is the **non-realizable case**.

Remark 1.3 (Non-Realizable Case)**.** A classification problem is **non-realizable** if no classifier $h \in \mathbb{H}_{mon}$ can achieve zero error on P, i.e., $k^* > 0$. This arises when the labels in P contain noise — points that violate monotonicity regardless of which classifier is chosen. This is analogous to the non-linearly separable case in SC2300.

In the context of Entity Matching, the non-realizable case is the norm. Furthermore, unlike the Perceptron, labels are **not** visible from the start — they are hidden and can only be revealed through probing. This is where **Active Classification** comes in. Devise a learning strategy where the algorithm selectively chooses which points to probe rather than requiring all labels upfront, gathering just enough evidence to construct a classifier with error at most $(1+\epsilon)k^*$ while minimizing the number of probes. In other words, Active Classification takes into account cost apart from just accuracy.

The contrast with passive classification is summarized below:

	Passive (Perceptron)	**Active (This Paper)**
Labels	All visible from start	Hidden, revealed via probes
Setting	Realizable ($k^* = 0$)	Non-realizable ($k^* > 0$)
Cost metric	Number of iterations	Number of probes
Goal	Zero training error	$(1+\epsilon)k^*$ error

Table 1: Passive vs. Active Classification

1.4.1 Why Monotone Classifiers are suitable for Active Learning

Monotone classification is particularly well-suited to this setting because it allows label propagation at no extra cost: if an oracle confirms that a pair p is a match ($+1$), then any pair q that is dominated by p (i.e., $q \preceq p$) cannot be a better match — so a monotone classifier must assign q a label $\leq +1$, meaning it is already constrained. Symmetrically, a confirmed non-match (-1) propagates upward. This means a single probe can resolve the labels of many other points for free, making monotone classification a natural and efficient framework for active learning in entity matching.

1.4.2 A Key Limitation of Active Classification

Existing active learning approaches require prior knowledge of k^* — the optimal monotone error — before the algorithm even begins. This is an unrealistic assumption in practice, since k^* can only be determined after all labels are revealed. This paper proposes algorithms that require no such prior knowledge, working instead by strategically probing points to implicitly converge towards a $(1+\epsilon)$-approximate solution — which we examine in the sections that follow.

2 RPE Algorithm

Having formulated the problem in Section 1.1, we now turn to our first algorithm for solving it: the Random Probes with Elimination (RPE) algorithm. This algorithm is the paper's primary contribution for the realizable case (k^*=0) and serves as the foundation for the more general coreset-based approach in Section 3.

2.1 The Algorithm

RPE repeatedly performs two steps:

1. Pick a random point and query its label

2. Use monotonicity to eliminate all points whose labels are now determined

algorithm RPE(P):

1. $Z = \emptyset$
2. **while** $P \neq \emptyset$ **do**
3. probe an element $z \in P$ chosen uniformly at random and add z to Z
4. **if** label$(z) = 1$ **then remove from** P every $p \in P$ satisfying $p \succeq z$
5. **else remove from** P every $p \in P$ satisfying $z \succeq p$
   /* note: z is removed by Line 4 or 5 */
6. **return** Z

Remark 2.1 (Intutition)**.** If a point z is positive, then every point above it must also be positive → remove them. If z is negative, then every point below it must also be negative → remove them. Thus, each probe "wipes out" a whole region of points.

The set Z returned by RPE is the set of all probed points. The classifier produced from Z is:

$$h_{\text{RPE}}(p) = \begin{cases} 1 & \text{if } \exists z \in Z \text{ such that label}(z) = 1 \text{ and } p \succeq z, \\ -1 & \text{otherwise.} \end{cases}$$

That is, a point p is classified as positive if it lies above any known positive point; otherwise it is classified as negative. Thus, the algorithm only needs to remember: "Where are the confirmed positive points?"

Why is the Classifier always Monotone?

Proof. Suppose for contradiction that there exist points $p \succ q$ such that $h_{\text{RPE}}(p) = -1$ and $h_{\text{RPE}}(q) = 1$. Since $h_{\text{RPE}}(q) = 1$, there exists some $z \in Z$ with label$(z) = 1$ such that $q \succeq z$. But then $p \succ q \succeq z$, so $p \succeq z$ as well. By the definition of h_{RPE}, this implies $h_{\text{RPE}}(p) = 1$, contradicting the assumption that $h_{\text{RPE}}(p) = -1$. □

Remark 2.2 (Intuition)**.** If a point q is labelled $+1$, it must dominate some positive probe z in Z. If another point p dominates q, then by transitivity ($p \succeq q \succeq z$), p also dominates that same positive probe z and must therefore also be $+1$.

2.2 Why It Works

The efficiency of RPE stems from the fact that each probe does "double duty". When RPE probes a point z and learns its label, monotonicity immediately determines the labels of an entire region points that are comparable to z.

- If label$(z) = 1$: any point $p \succeq z$ is at least as similar as z on every feature. Monotonicity forces $h(p) \geq h(z) = 1$, so all such p must be labelled $+1$ and can be removed from P.
- If label$(z) = -1$: any point $p \preceq z$ is dominated by z, so monotonicity forces $h(p) \leq h(z) = -1$. All such p must be labelled -1 and are removed.

Only points whose label cannot yet be inferred by monotonicity remain in P after each elimination step. These are the points the algorithm must still resolve, and it will probe them in subsequent iterations.

2.3 A Worked Example: RPE in Action

We trace RPE on the 2D point set P from Figure 2. Recall that black points have label $+1$ and white points have label -1. The optimal monotone classifier achieves $k^* = 3$ by misclassifying only p_1, p_{11}, p_{15}. The dominance width of this input is $w = 6$.

Example 2.1 (from paper, Example 1.3)**.** Suppose RPE randomly probes p_1 first.

- **Step 1:** Probe p_1. The oracle reveals $\text{label}(p_1) = 1$. RPE removes every $p \succeq p_1$. Since p_1 sits towards the bottom-left of the figure, it dominates (or equals) almost all other points. The entire P is eliminated except p_6, p_7, p_8 (which lie below or incomparable to p_1).
- **Step 2:** Probe p_8. The oracle reveals $\text{label}(p_8) = -1$. RPE removes every $p \preceq p_8$, which eliminates p_6, p_7 and p_8 itself. P is now empty. RPE terminates with $Z = \{p_1, p_8\}$.
- **Classifier output:** Since $\text{label}(p_1) = 1$, every point $p \succeq p_1$ is mapped to $+1$ by h_{RPE}. All other points (including p_6, p_7, p_8) are mapped to -1.
- **Error:** $\text{err}_P(h_{\text{RPE}}) = 5$ (5 white points mapped to $+1$ due to dominating p_1).

This example demonstrates how a single probe of p_1 is enough to determine the labels of 13 out of 16 points. The cost of 2 probes is far below the naïve cost of $n = 16$. Note, however, that the error of 5 exceeds $k^* = 3$ — we will see why this is expected and bounded by the $2k^*$ guarantee.

2.4 Guarantees

The following theorem (proved in full in the paper) captures the performance of RPE:

Theorem 2.1 (RPE Guarantees)**.** For Problem 1.1, the RPE algorithm satisfies:

- **Expected error:** The classifier h_{RPE} has expected error at most $2k^*$:

$$\mathbb{E}[\text{err}_P(h_{\text{RPE}})] \leq 2k^*.$$

- **Expected cost:** RPE probes at most $O(w \cdot \log(n/w))$ elements in expectation, where w is the dominance width of P, $n = |P|$, and $\log(x) = \log_2(1 + x)$.

Two aspects of this theorem are worth emphasizing. First, RPE never needs to compute w — the width is only used in the analysis. Second, when $k^* = 0$ (the realizable case), the expected error guarantee of $2k^* = 0$ means RPE always finds an optimal classifier.

Proof Sketch

1. Error Analysis

Fix any monotone classifier $h \in \mathbb{H}_{\text{mon}}$. Call a point $p \in P$ *h-good* if $h(p) = \text{label}(p)$ and *h-bad* otherwise. The proof proceeds by induction on $|P|$, showing:

Lemma 2.1. The number of *h-good* elements misclassified by h_{RPE} is at most $\text{err}_P(h)$ in expectation.

Setting $h = h^*$ (the optimal classifier), there are exactly k^* h^*-bad elements. The total expected error is thus $\leq k^* + k^* = 2k^*$.

The central tool in the inductive argument is a **pairing between influence sets**. When RPE probes a point z, it may cause other points to be misclassified:

- For an h-bad point p, $I_{\text{bad}}(p)$ is the set of h-good points that get forced into the wrong label if p is probed first.
- For an h-good point q, $I_{\text{good}}(q)$ is the set of h-bad points that get forced into the correct label (according to h) if q is probed first.

The key insight is that these influences balance perfectly:

Lemma 2.2. For any h-bad $p \in P$ and h-good $q \in P$, $q \in I_{\text{bad}}(p) \iff p \in I_{\text{good}}(q)$. Hence, $\sum_{p \in \text{bad}} |I_{\text{bad}}(p)| = \sum_{q \in \text{good}} |I_{\text{good}}(q)|$.

Why does this give the $2k^*$ bound on the total expected error? Because the total "potential for error" (left side) equals the total "potential for correction" (right side), the expected number of h-good points misclassified by RPE is bounded by the number of h-bad points (k^*). Adding the k^* bad points themselves (which are always misclassified) yields:

$$E[\text{err}_P(h_{\text{RPE}})] \leq k^* + k^* = 2k^*.$$

Example 2.2. Let P be the set of points in Figure 2, and h be the classifier that maps all the black points to 1 except p_1 and all the white points to -1 except p_{11} and p_{15}. Thus, p_1, p_{11}, and p_{15} are h-bad, while the other points of P are h-good. The following are some representative influence sets:

- $I_{\text{bad}}(p_{15}) = \{p_4, p_9, p_{10}, p_{12}, p_{13}, p_{14}, p_{16}\}$
- $I_{\text{bad}}(p_1) = \{p_2, p_3, p_5\}$
- $I_{\text{good}}(p_3) = \{p_1\}$
- $I_{\text{good}}(p_9) = \{p_{11}, p_{15}\}$

2. Cost Analysis

The efficiency of RPE is not measured by the total points n, but by the dominance width w (the size of the largest set of incomparable points). The proof is summarized as follows:

1. **Decomposition**: By Dilworth's Theorem (Section 2.7.1), any point set P can be partitioned into w disjoint chains (where every point in a chain is comparable to the others).
2. **Parallel Search**: RPE effectively performs a randomized search on all w chains simultaneously. Each probe in P is guaranteed to land in one of these chains.
3. **A&E Game**: Within any single chain, RPE's behavior mimics an "Attrition-and-Elimination" game (Section 2.7.2). Even if an adversary removes points to try and trick the algorithm, a random probe is likely to "hit" near the middle of the remaining chain, eliminating half of it.

By summing the expected probes across w chains and applying the concavity of the logarithm (Jensen's Inequality), we arrive at the total expected cost: $O(w \log \frac{n}{w})$.

2.5 Another Worked Example: Why the Factor of 2 Is Tight

The error guarantee of $2k^*$ is the *best possible* approximation ratio for RPE. The following example demonstrates this:

Example 2.3. Consider Figure 3, where a point set P in $\mathbb{R}^2$ contains one black point (label $+1$) and $n-1$ white points (label -1).

Optimal error: The monotone classifier that maps all points to -1 misclassifies only the black point, so $k^* = 1$.

RPE's expected error: With probability $1/n$, RPE probes the black point first. Since it has label $+1$ and dominates nothing, no points are eliminated. So, all $n-1$ white points remain and are later mapped to $+1$ by h_{RPE}, giving error $n-1$. With probability $1-1/n$, RPE probes a white point first, eliminates all dominated points (just itself), and the resulting error is 1. Thus:

$$\mathbb{E}[\text{err}(h_{\text{RPE}})] = \frac{1}{n}(n-1) + \left(1 - \frac{1}{n}\right)(1) = 2 - \frac{2}{n} \to 2 \quad \text{as } n \to \infty.$$

Since $k^* = 1$, the approximation ratio approaches $2k^*/k^* = 2$. The factor of 2 is therefore *asymptotically tight* for RPE.

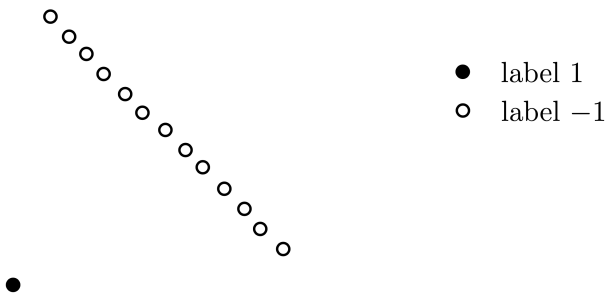

Figure 3: The approximation ratio 2 is tight for RPE.

2.6 Limitations

RPE has two key limitations:

1. **The factor-of-2 error is tight.** As shown in Example 2.3, no modification to RPE's random-probing strategy can reduce the approximation ratio below 2. To achieve a ratio of $1+\epsilon$ for arbitrary $\epsilon > 0$, a fundamentally different technique is needed. This is the motivation for the *relative-comparison coreset* approach in Section 3.

2. **No high-probability guarantee.** Theorem 2.1 gives an *expected* error bound. In a single run, RPE may produce an error much larger than $2k^*$, as the bad scenario in Example 2.3 illustrates. The coreset-based algorithm upgrades this to a *w.h.p.* guarantee at the cost of an extra $O(\log n/\epsilon^2)$ factor in probes.

Table 2: RPE vs. Coreset Algorithm

Property	**RPE (Section 2)**	**Coreset Alg. (Section 3)**
Error guarantee	$\mathbb{E}[\text{err}] \le 2k^*$ (expected)	$(1+\epsilon)k^*$ (w.h.p.)
Approx. ratio	2 (tight)	$1+\epsilon$ (any $\epsilon > 0$)
Probing cost	$O(w \cdot \log(n/w))$	$O(\frac{w}{\epsilon^2} \cdot \log(n/w) \cdot \log n)$
Requires k^*?	No	No
High-probability?	No	Yes

2.7 Chains, Dilworth's Theorem, and the A&E Game (Supplementary)

This section covers supplementary material relevant to the cost analysis.

2.7.1 Chains and Dilworth's Theorem

A *chain* is a subset $S \subseteq P$ where every two elements are comparable under dominance: we can order them as $p_1 \preceq p_2 \preceq \cdots \preceq p_{|S|}$. A *chain decomposition* partitions P into disjoint chains. Dilworth's theorem [**?**] guarantees the existence of a chain decomposition using exactly w chains (where w is the width of P).

Example 2.4. The input P in Figure 2 can be decomposed into 6 chains:

- $C_1 = \{p_1, p_2, p_3, p_4, p_{10}\}$
- $C_2 = \{p_{11}\}$
- $C_3 = \{p_5, p_9, p_{12}\}$

- $C_4 = \{p_{16}\}$
- $C_5 = \{p_{13}\}$
- $C_6 = \{p_6, p_7, p_8, p_{14}, p_{15}\}$

The points $p_{10}, p_{11}, p_{12}, p_{16}, p_{13}, p_{14}$ indicate the absence of any chain decomposition of P having less than 6 chains.

2.7.2 A&E Game

To analyze RPE's cost within a single chain, the paper introduces a two-player *Attrition-and-Elimination (A&E) game* on a chain C of m points. In each round:

- Bob (adversary) may delete any subset of points from C (attrition).
- Alice picks a uniformly random point $p \in C$ and deletes all points $q \succeq p$ (elimination).

The game ends when C is empty. This generalizes a **random binary search**: while a standard binary search assumes the points stay put, the A&E game proves that even if an adversary removes points to "hide" the middle, Alice will still finish quickly.

Lemma 2.3. Regardless of Bob's strategy, the expected number of rounds Y satisfies $\mathbb{E}[Y] = O(\log m)$.

The proof proceeds in two steps: showing that each round has a high probability of being "successful," and then bounding the total rounds needed.

1. Probability of a "Successful" Round: Define a round as *successful* if it reduces the number of elements in the chain by at least a factor of 2.

- Let m' be the number of points left after Bob's attrition in a round.
- Let the points be ordered $p_1 \preceq p_2 \preceq \cdots \preceq p_{m'}$.
- If Alice picks any point p_i where $i \leq \frac{m'}{2} + 1$, then at most half the points remain after she eliminates everything dominating p_i.
- Since Alice picks uniformly at random, the probability of picking such a point is $> 1/2$. Thus, every round is successful with probability $> 1/2$, regardless of what Bob does.

2. Total Expected Rounds: A chain of size m can only be halved $\log_2(m)$ times before it is empty. Therefore, we only need $\log_2(m)$ "successful" rounds to end the game.

- If we run $Y = 2\log_2(m)$ rounds, we expect half of them to be successful (which would be exactly $\log_2(m)$).
- Using the properties of independent trials, the probability that we fail to reach $\log_2(m)$ successes in $2\log_2(m)$ rounds is very small (less than $1/m$).
- Even in the worst-case scenario where we never get enough successes, the game must end in at most m rounds (since Alice removes at least one point per round).

Combining these, the expected number of rounds is dominated by the $2\log_2(m)$ term:

$$\mathbb{E}[Y] \leq 2\log_2(m) + m \cdot \mathbf{Pr}(\text{failure}) = O(\log m).$$

3 Relative-Comparison Coresets (RCC)

3.1 Why RPE Isn't Enough

Recall that RPE achieves an expected error of at most $2k^*$, and this guarantee is tight. In the entity-matching context a factor-2 margin may be unacceptable for a production pipeline.

The goal of this section is to achieve an approximation ratio of $(1+\varepsilon)$ for any user-specified $\varepsilon > 0$: find a function $F : \mathbb{H}_{\text{mon}} \to \mathbb{R}$ such that

$$F(h) \le F(h') \implies \text{err}_P(h) \le (1+\varepsilon)\,\text{err}_P(h').$$

Since $F(\hat{h}) \le F(h^*)$ for any optimal h^*, this gives $\text{err}_P(\hat{h}) \le (1+\varepsilon)k^*$.

A natural approach — estimating $\text{err}_P(h)$ directly by probing — fails: even for the all-$+1$ classifier h^{pos}, estimating its error to within a constant relative factor requires locating the single -1 element (or confirming none exists), which costs $\Omega(n)$ probes in expectation.

3.2 The Unknown-Δ Technique

We sidestep this barrier by constructing a surrogate $F : \mathbb{H}_{\text{mon}} \to \mathbb{R}$ satisfying, for all $h \in \mathbb{H}_{\text{mon}}$, a *shifted* approximation with some unknown constant Δ:

$$\text{err}_P(h)\big(1-\tfrac{\varepsilon}{4}\big) + \Delta \;\le\; F(h) \;\le\; \text{err}_P(h)\big(1+\tfrac{\varepsilon}{4}\big) + \Delta \qquad \forall\, h \in \mathbb{H}_{\text{mon}}, \tag{1}$$

Proposition 3.1. Any F satisfying (1) has the relative ε-comparison property.

Proof. If $F(h) \le F(h')$, apply the left bound of (1) to h and the right bound to h', then cancel Δ:

$$\text{err}_P(h)\big(1-\tfrac{\varepsilon}{4}\big) \;\le\; F(h) - \Delta \;\le\; F(h') - \Delta \;\le\; \text{err}_P(h')\big(1+\tfrac{\varepsilon}{4}\big)\,.$$

Rearranging and using $\frac{1+\varepsilon/4}{1-\varepsilon/4} \le 1+\varepsilon$ for $\varepsilon \in (0,1]$ gives the result. □

The key insight: Δ cancels whenever we *compare* two classifiers. We never need to know its value — only its existence.

3.3 The Coreset Technique

We realise F via a **relative-comparison coreset**: a weighted subset $Z \subseteq P$ whose weighted error $\text{w-err}_Z(h)$ satisfies (1) for all h.

Definition 3.1 (Relative-Comparison Coreset)**.** A **relative-comparison coreset** of P is a subset $Z \subseteq P$ where every $z \in Z$ has its label revealed and carries a positive weight $\text{weight}(z)$, such that the *weighted error*

$$\text{w-err}_Z(h) \;:=\; \sum_{z \in Z : h(z) \neq \text{label}(z)} \text{weight}(z)$$

satisfies (1) for every $h \in \mathbb{H}_{\text{mon}}$ — i.e., w-err_Z serves as the desired function F.

The novelty of the unknown-Δ approach is that classical coresets approximate $\text{err}_P(h)$ itself (costing $\Omega(n)$ probes), whereas here Δ never needs to be computed.

3.4 Building the Coreset in 1D: The Recursive Framework

Setup. The coreset is built by BuildCoreset (Algorithm 1), which recursively solves the following on each subproblem $Q \subseteq P$: find $F : \mathbb{H}_{\text{mon}} \to \mathbb{R}$ satisfying both

$$|F(h) - \text{err}_Q(h)| \;\le\; \varepsilon|Q|/64, \tag{2}$$

$$\text{err}_Q(h)(1 - \tfrac{\varepsilon}{4}) + \Delta \;\le\; F(h) \;\le\; \text{err}_Q(h)(1 + \tfrac{\varepsilon}{4}) + \Delta, \tag{3}$$

for some $|\Delta| \le \varepsilon|Q|/64$.

Finitising $\mathbb{H}_{\text{mon}}$. Although $\mathbb{H}_{\text{mon}}$ is infinite, only the $|P|+1$ thresholds that fall on or between points of P produce distinct labellings; taking the union bound over this finite set $\mathbb{H}^P_{\text{mon}}$ is what introduces the $\log|P|$ factor in the sample size m.

Algorithm 1 BuildCoreset(P)

```
if |P| = 1 then
    Probe p ∈ P; add p to Z with weight(p) = 1.
else
    Sample S_1 of size m from P; use G_1 to find α, β and partition P into P_α, P_mid, P_rest.
    if P_rest ≠ ∅ then
        Sample S_2 of size m from P_rest; add S_2 to Z with weight |P_rest|/|S_2|.
    end if
    if P_α ≠ ∅ then
        Sample S_α of size m from P_α; add S_α to Z with weight |P_α|/|S_α|.
    end if
    Z ← Z ∪ BuildCoreset(P_mid)
end if
return Z
```

3.4.1 Estimating Error via Sampling (Functions G_1, G_2, F_α)

At each recursion level on a set $Q \subseteq P$, we draw a uniform sample S of size $O\Big(\frac{1}{\varepsilon^2}\log\frac{|Q|\,\ell}{\delta}\Big)$ from Q, where $\ell = O(\log n)$ is the number of recursion levels. Define

$$G(h) \;:=\; \frac{|Q|}{|S|} \cdot \text{err}_S(h).$$

By a standard concentration bound, G satisfies condition (2) for Q simultaneously for all $h \in \mathbb{H}_{\text{mon}}$, with high probability. Three such estimates are used per level:

- G_1, sampled from the full P, is used to *locate the split point*. Does not enter the coreset.
- G_2, sampled from P_{rest}, approximates $\text{err}_{P_{\text{rest}}}(h)$ and contributes its sampled points to Z.
- F_α, sampled from P_α, approximates $\text{err}_{P_\alpha}(h)$ and contributes its sampled points to Z.

Together, the target function for the problem in 1D is,

$$F(h) = G_2(h) + F_\alpha(h) + F_{\text{mid}}(h)$$

where F_{mid} comes from recursion on P_{mid} satisfies both conditions 2 and 3 for the full P.

As there are ℓ levels, the overall cost is $O\Big(\frac{\ell}{\epsilon^2}\log\frac{n}{\delta}\Big) = O\Big(\frac{\log n}{\epsilon^2}\cdot\log\frac{n}{\delta}\Big)$, which lets us solve the 1D problem with a probability of at least $1-\delta$.

3.4.2 Splitting P into Three Parts

The purpose of G_1 is to identify where the error function transitions through the value $|P|/4$. Concretely, define:

$$\alpha = \text{smallest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\tfrac{1}{4} - \tfrac{\varepsilon}{64}\right),$$
$$\beta = \text{largest } \tau \in P \cup \{-\infty\} \text{ with } G_1(h^\tau) < |P|\left(\tfrac{1}{4} - \tfrac{\varepsilon}{64}\right).$$

Because G_1 satisfies (2), any h^τ with $G_1(h^\tau)$ below this threshold has $\mathrm{err}_P(h^\tau) < |P|/4$. This splits P into: $P_\alpha = \{p \in P : p = \alpha\}$, $P_{\mathrm{mid}} = \{p \in P : \alpha < p \le \beta\}$, $P_{\mathrm{rest}} = P \setminus (P_\alpha \cup P_{\mathrm{mid}})$.

Proposition 3.2 (Size of P_{mid})**.** $|P_{\mathrm{mid}}| < |P|/2 \to$ Recursion terminates in $O(\log n)$.

Proof. If $\alpha = \beta =$ null, the claim is trivial. Otherwise, P_{mid} cannot contain $\ge |P|/4$ positive-label points (else $G_1(h_\beta) \ge |P|(1/4 - \varepsilon/64)$, contradicting the definition of β). By a symmetric argument, it cannot contain $\ge |P|/4$ negative-label points either. Hence $|P_{\mathrm{mid}}| < |P|/2$. □

3.4.3 Assembling the Coreset Z

- A sample S_2 drawn from P_{rest} enters Z with weight $|P_{\mathrm{rest}}|/|S_2|$.
- A sample S_α drawn from P_α enters Z with weight $|P_\alpha|/|S_\alpha|$.
- The recursion on P_{mid} returns a sub-coreset Z_{mid}, which is also included in Z.

Setting $F(h) = G_2(h) + F_\alpha(h) + F_{\mathrm{mid}}(h)$ and $\Delta = \Delta_\alpha + \Delta_{\mathrm{mid}} + (G_2(h_\beta) - \mathrm{err}_{P_{\mathrm{rest}}}(h_\beta))$, one can verify that the resulting F satisfies both (2) and (3) for Q, with $\Delta = \Delta_\alpha + \Delta_{\mathrm{mid}} + (G_2(h_\beta) - \mathrm{err}_{Q_{\mathrm{rest}}}(h_\beta))$.

3.5 Extension to Higher Dimensions

For $d > 1$, compute a chain decomposition $C_1, \ldots, C_w$ of P via Dilworth's Theorem (the same decomposition used in the RPE analysis). Apply the 1D construction independently to each chain, yielding sub-coresets $Z_1, \ldots, Z_w$. The final coreset $Z = \bigcup_i Z_i$ is valid because error decomposes additively over chains:

$$\mathrm{err}_P(h) = \sum_{i=1}^{w} \mathrm{err}_{C_i}(h), \qquad \text{w-err}_Z(h) = \sum_{i=1}^{w} \text{w-err}_{Z_i}(h), \qquad \Delta = \sum_i \Delta_i.$$

3.6 Main Results

Theorem 3.1 (Theorem 6 in the paper)**.** Let n and w be the size and width of the input P, respectively. In $O\left(\frac{w}{\varepsilon^2}\log\frac{n}{w} \cdot \log\frac{n}{\delta}\right)$ probes, we can obtain with probability at least $1-\delta$ a relative-comparison coreset Z of P with $|Z| = O\left(\frac{w}{\varepsilon^2}\log\frac{n}{w} \cdot \log\frac{n}{\delta}\right)$.

Given the coreset Z, finding the best monotone classifier requires *no further probing*: simply compute $\hat{h} = \arg\min_{h \in \mathbb{H}_{\mathrm{mon}}} \text{w-err}_Z(h)$, which can be done in time polynomial in $|Z|$ and d.

Corollary 3.1 (Corollary 7 in the paper)**.** For Problem 1 with any $\varepsilon \in (0, 1]$, there exists an algorithm that finds, with high probability, a monotone classifier with error at most $(1+\varepsilon)k^*$

using $O\Big(\frac{w}{\varepsilon^2}\log\frac{n}{w}\cdot\log n\Big)$ probes, where $n=|P|$, w is the width of P, and k^* is the optimal monotone error.

The argument is direct: for any $h\in\mathbb{H}_{\text{mon}}$, the coreset bound gives

$$\text{err}_P(\hat{h})(1-\tfrac{\varepsilon}{4})\le \text{w-err}_Z(\hat{h})-\Delta\le \text{w-err}_Z(h^*)-\Delta\le \text{err}_P(h^*)(1+\tfrac{\varepsilon}{4})\,,$$

so setting $h=h^*$ (the true optimal) and rearranging yields $\text{err}_P(\hat{h})\le(1+\varepsilon)k^*$.

3.7 Example

Let $P=\{1,\dots,8\}$ with $\ell(p)=-1$ for $p\le 4$ and $\ell(p)=+1$ for $p\ge 5$, $\varepsilon=\frac{1}{2}$. Classifiers have the form $h^\tau(p)=+1$ iff $p>\tau$; the optimal is h^4 with $k^*=0$.

Level 1: BuildCoreset(P)

Draw $S_1=\{2,5,7\}$, scaling factor $|P|/|S_1|=8/3$. Threshold: $|P|(1/4-\varepsilon/64)=8\times 31/128\approx 1.94$.

h^τ	err_P	err_{S_1}	$G_1=\frac{8}{3}\cdot\text{err}_{S_1}$	$G_1<1.94$?
h^{pos}	4	1	2.67	No
h^1	3	1	2.67	No
h^2	2	0	0	**Yes**
h^3	1	0	0	**Yes**
h^4	0	0	0	**Yes**
h^5	1	1	2.67	No
h^6	2	2	5.33	No
h^7	3	2	5.33	No
h^8	4	2	5.33	No

$\alpha=2$, $\beta=4$, giving partition $P_\alpha=\{2\}$, $P_{\text{mid}}=\{3,4\}$, $P_{\text{rest}}=\{1,5,6,7,8\}$ ($|P_{\text{mid}}|=2<4$ ✓). Sample $S_2=\{6,7\}$ from P_{rest} with weight $5/2$; sample $S_\alpha=\{2\}$ from P_α with weight 1. Recurse on P_{mid}.

Level 2: BuildCoreset(P_{mid})

$P_{\text{mid}}=\{3,4\}$ (both labelled -1). Draw $S_1=\{4\}$, scaling factor $2/1=2$. Threshold: $2\times 31/128\approx 0.484$.

h^τ	$\text{err}_{P_{\text{mid}}}$	err_{S_1}	$G_1=2\cdot\text{err}_{S_1}$	$G_1<0.484$?
h^2	2	1	2	No
h^3	1	1	2	No
h^4	0	0	0	**Yes**

$\alpha=\beta=4$, giving $P_\alpha=\{4\}$, $P'_{\text{mid}}=\emptyset$, $P_{\text{rest}}=\{3\}$. Both subsets are singletons: $S_2=\{3\}$ and $S_\alpha=\{4\}$, each with weight 1. Recursion terminates ($P'_{\text{mid}}=\emptyset$).

Final Coreset Z and Weighted Error

Point	Label	Weight
2	-1	1
3	-1	1
4	-1	1
6	$+1$	5/2
7	$+1$	5/2
Total		$1+1+1+\frac{5}{2}+\frac{5}{2}=8=\lvert P\rvert$ ✓

h^τ	w-err_Z	h^τ	w-err_Z
h^{pos}	3	h^5	0
h^1	3	h^6	5/2
h^2	2	h^7	5/2
h^3	1	h^{neg}	5
h^4	**0**		

h^4 (and h^5) minimise w-err_Z, correctly recovering $h^*=h^4$.

4 Proofs and Lower Bounds

Having established algorithms for monotone classification (RPE achieving $2k^*$ in Section 2 and the coreset-based algorithm achieving $(1+\varepsilon)k^*$ in Section 3), the natural question is: *can we do better?* This section answers that question by proving **lower bounds** — results showing that *no algorithm* can substantially outperform the ones we have. Together with the upper bounds, these lower bounds establish that our algorithms are **near-optimal**.

We proceed in three stages, mirroring the three regimes of ε:

1. **Exact case** ($\varepsilon = 0$): finding an optimal classifier requires $\Omega(n)$ probes (Theorem 4.1).
2. **Constant approximation** (ε is a constant): any c-approximate algorithm requires $\Omega(w \cdot \text{Log}\, \frac{n}{(k^*+1)w})$ probes (Theorem 4.2).
3. **Arbitrary** ε: any $(1+\varepsilon)$-approximate algorithm requires $\Omega(w/\varepsilon^2)$ probes (Theorem 4.3).

4.1 The Exact Case: $\varepsilon = 0$ Requires $\Omega(n)$ Probes

When $\varepsilon = 0$, the algorithm must find an *optimal* monotone classifier — one that achieves exactly k^* errors. The following theorem shows this is essentially as hard as reading the entire input.

Theorem 4.1 (Theorem 10 in the paper)**.** For Problem 1.1, any algorithm that finds an optimal monotone classifier with probability greater than $2/3$ must probe $\Omega(n)$ elements in expectation. This holds even when:

- the dimensionality $d = 1$, and
- the algorithm knows k^* in advance.

Remark 4.1 (Significance)**.** This result justifies the entire study of the approximate case ($\varepsilon > 0$). If finding the exact optimum were cheap, there would be no need for approximation. The $\Omega(n)$ lower bound shows that *even with strong prior knowledge* (knowing k^*), no clever probing strategy can avoid reading nearly all of P.

4.1.1 The Hard Input Family

The proof constructs a family $\mathcal{P}$ of n one-dimensional inputs that any algorithm must distinguish between. All inputs share the same point locations $\{1, 2, \ldots, n\}$ and differ only in their label assignments.

Construction. Group the n elements into $n/2$ *pairs*: $(1,2), (3,4), \ldots, (n-1, n)$. In a *normal pair* $(2i-1, 2i)$, element $2i-1$ has label $+1$ and element $2i$ has label -1. Each input in $\mathcal{P}$ contains exactly one *anomaly pair* where the normal labelling is violated:

- (-1)**-input** $P_{-1}(i)$: The anomaly pair is $(2i-1, 2i)$ where *both* elements receive label -1. That is, element $2i-1$ is flipped from $+1$ to -1.
- $(+1)$**-input** $P_{+1}(i)$: The anomaly pair is $(2i-1, 2i)$ where *both* elements receive label $+1$. That is, element $2i$ is flipped from -1 to $+1$.

This gives $n/2$ choices for the anomaly pair and 2 choices for its type, yielding $|\mathcal{P}| = n$ total inputs.

Example 4.1. For $n = 6$, the elements are $\{1, 2, 3, 4, 5, 6\}$ grouped into pairs $(1, 2), (3, 4), (5, 6)$. In a normal pair, labels are $(+1, -1)$. Some inputs in $\mathcal{P}$:

Input	1	2	3	4	5	6	Anomaly
$P_{-1}(1)$	$\mathbf{-1}$	-1	$+1$	-1	$+1$	-1	pair 1 flipped to $(-1, -1)$
$P_{+1}(2)$	$+1$	-1	$+1$	$\mathbf{+1}$	$+1$	-1	pair 2 flipped to $(+1, +1)$
$P_{-1}(3)$	$+1$	-1	$+1$	-1	$\mathbf{-1}$	-1	pair 3 flipped to $(-1, -1)$

Bold entries mark the flipped element that creates the anomaly.

4.1.2 Key Properties of the Family

Proposition 4.1 (Proposition 5 in the paper)**.** For any $i \in [n/2]$, no monotone classifier can be optimal for *both* $P_{-1}(i)$ and $P_{+1}(i)$.

Proof. Every input in $\mathcal{P}$ has optimal monotone error $k^* = n/2 - 1$. Every 1D monotone classifier has the form h^τ (mapping points $> \tau$ to $+1$). Consider the three cases for a classifier h^τ:

- $\tau < 2i - 1$: On $P_{-1}(i)$, both elements $2i - 1$ and $2i$ are labelled -1 but h^τ maps them to $+1$, giving error $n/2 + 1 > k^*$.
- $\tau = 2i - 1$: On $P_{-1}(i)$, element $2i$ is labelled -1 but mapped to $+1$, giving error $n/2 > k^*$.
- $\tau \geq 2i$: On $P_{+1}(i)$, both elements $2i - 1$ and $2i$ are labelled $+1$ but h^τ maps them to -1, giving error $n/2 + 1 > k^*$.

In every case, h^τ is suboptimal on at least one of $P_{-1}(i)$ or $P_{+1}(i)$. □

4.1.3 The Counting Argument (Lemma 11)

To prove the lower bound, the paper first analyzes *deterministic* algorithms, then extends to randomized ones via Yao's minimax theorem.

Free labels boost. To make the lower bound stronger, we *help* the algorithm: whenever it probes one element of a pair $(2i - 1, 2i)$, we reveal the label of the other element for free. If the lower bound holds even with this advantage, it must hold without it. We say the algorithm "probes pair i" when it probes either $2i - 1$ or $2i$.

A deterministic algorithm A_{det} (with free labels) probes pairs in a fixed sequence $x_1, x_2, \ldots, x_t$. It stops when it discovers the anomaly pair (since all normal pairs look identical). If it reaches the end of the sequence without finding the anomaly, it outputs a fixed classifier h_{det}.

Lemma 4.1 (Lemma 11 in the paper)**.** Fix any deterministic algorithm A_{det} and constant $0 \leq c < 1$. For $n \geq 4$: if A_{det} errs on at most $cn/2$ inputs in $\mathcal{P}$, then its total cost across all inputs in $\mathcal{P}$ (denoted family-cost) satisfies:

$$\text{family-cost}(A_{\text{det}}) \;\geq\; \frac{n^2(1 - c^2)}{4}.$$

Proof. Define:

$$\text{family-err}(A_{\text{det}}) = \text{number of inputs in } \mathcal{P} \text{ on which } A_{\text{det}} \text{ fails to find an optimal classifier,}$$
$$\text{family-cost}(A_{\text{det}}) = \text{total number of probes across all inputs in } \mathcal{P}.$$

Step 1: Error counting. For each $i \notin \{x_1, \ldots, x_t\}$, the algorithm never probes pair i, so it outputs the same classifier h_{det} on both $P_{-1}(i)$ and $P_{+1}(i)$. By Proposition 4.1, h_{det} must fail on at least one of them. Thus:

$$\text{family-err}(A_{\text{det}}) \;\geq\; n/2 - t.$$

Step 2: Cost accounting. For each $i \notin \{x_1, \ldots, x_t\}$, the algorithm probes all t pairs on both $P_{-1}(i)$ and $P_{+1}(i)$, contributing $2t$ probes. For $i = x_j$ (i.e., the anomaly is found at position j), the algorithm probes j pairs on each of $P_{-1}(x_j)$ and $P_{+1}(x_j)$, contributing $2j$ probes. Hence:

$$\text{family-cost}(A_{\text{det}}) = 2t \cdot (n/2 - t) + 2\sum_{j=1}^{t} j = nt - t^2 + t.$$

Step 3: Combining. If family-err $\leq cn/2$, then $t \geq n(1-c)/2$. For $t \in [n(1-c)/2,\ n/2]$:

$$\text{family-cost} \;\geq\; nt - t^2 \;\geq\; \frac{n^2(1-c^2)}{4}.$$ □

4.1.4 From Deterministic to Randomized (Corollary 12)

Corollary 4.1 (Corollary 12 in the paper)**.** For $n \geq 4$, any randomized algorithm A with family-err$(A) < n/3$ satisfies $\mathbf{E}[\text{family-cost}(A)] = \Omega(n^2)$.

Proof. A randomized algorithm is a distribution over deterministic algorithms. Call a deterministic algorithm A_{det} *accurate* if family-err$(A_{\text{det}}) \leq 2n/5$.

Claim: $\mathbf{Pr}[A \in A_{\text{acc}}] > 1/6$. Otherwise, $\mathbf{Pr}[A \notin A_{\text{acc}}] \geq 5/6$, and since inaccurate algorithms err on $> 2n/5$ inputs:

$$\text{family-err}(A) \;\geq\; \frac{2n}{5} \cdot \mathbf{Pr}[A \notin A_{\text{acc}}] \;\geq\; \frac{2n}{5} \cdot \frac{5}{6} = \frac{n}{3},$$

contradicting family-err$(A) < n/3$.

By Lemma 4.1 with $c = 4/5$, every accurate algorithm has family-cost $= \Omega(n^2)$, so:

$$\mathbf{E}[\text{family-cost}(A)] \;\geq\; \Omega(n^2) \cdot \mathbf{Pr}[A \in A_{\text{acc}}] = \Omega(n^2).$$

□

Remark 4.2 (How the Corollary implies Theorem 4.1)**.** If algorithm A finds an optimal classifier with probability $> 2/3$ on *every* input, then family-err$(A) < |\mathcal{P}|/3 = n/3$. By Corollary 4.1, $\mathbf{E}[\text{family-cost}(A)] = \Omega(n^2)$. Since $|\mathcal{P}| = n$, by an averaging argument at least one input in $\mathcal{P}$ requires $\Omega(n)$ expected probes.

4.2 Lower Bound for Constant Approximation Ratios

Having shown that exact optimality is expensive, we now ask: does allowing a constant approximation ratio (e.g., $c = 2$ or $c = 10$) significantly reduce the cost? The answer is: it helps, but there is still a non-trivial lower bound that scales with the *width* w.

Theorem 4.2 (Theorem 13 in the paper)**.** Let n', w', k, c be arbitrary integers satisfying $n' \geq 2, w' \geq 1, k \geq 0, c \geq 1$, and n' is a multiple of w'. Set $n = n' + 2k + 2ckn'$ and $w = w' + \mathbf{1}_{k \geq 1}$. For Problem 1.1, there exists a family $\mathcal{P}$ of inputs with size n, width w, and optimal monotone error $k^* = k$ such that any randomized algorithm guaranteeing an expected error $\leq c \cdot k^*$ must probe $\Omega(w' \cdot \text{Log}\, \frac{n'}{w'})$ elements in expectation on at least one

input.

Remark 4.3 (Special cases)**.**

- **Realizable case** ($k = 0$): The theorem gives a lower bound of $\Omega(w \cdot \text{Log}\, \frac{n}{w})$, matching the RPE upper bound up to constants.
- **Non-realizable case** ($k \geq 1$): Using $n'/w' \geq n/(4ckw)$, the lower bound becomes $\Omega(w \cdot \text{Log}\, \frac{n}{k^* w})$.

4.2.1 Proof Strategy

The proof proceeds in two stages.

Stage 1: The realizable case ($k = 0$). We construct a hard family using w' mutually independent boxes in $\mathbb{R}^2$. Each box contains n'/w' points on its main diagonal. Labels are assigned independently per box: for each box, we choose one of $1+n'/w'$ possible label assignments (specifying how many bottom points get label -1). This yields a family of $(1 + n'/w')^{w'}$ inputs.

Since any algorithm that always finds an optimal classifier must distinguish all these inputs, its decision tree must have at least $(1 + n'/w')^{w'}$ leaves. A binary tree with that many leaves has average depth $\Omega(w' \cdot \text{Log}\, \frac{n'}{w'})$, establishing the lower bound for deterministic algorithms. Yao's minimax theorem extends it to randomized ones.

Stage 2: Reduction from non-realizable to realizable. For $k \geq 1$, we "pad" each realizable input with *dummy points* to create a non-realizable instance with $k^* = k$.

Definition 4.1 (Dummy Points Construction)**.** Given a realizable input P' with w' boxes:

1. For each box B_i and each pair of consecutive points p_j, p_{j+1} in B_i:
   - If $\text{label}(p_j) = \text{label}(p_{j+1})$: insert $2ck$ dummy points with the same label.
   - If $\text{label}(p_j) = -1$ and $\text{label}(p_{j+1}) = +1$ (label transition): insert ck dummies with label -1 and ck dummies with label $+1$.
2. Add a *dummy box* independent from all others, containing $2k$ points: the bottom k have label $+1$ and the top k have label -1 (deliberately reversed to force k errors).

Proposition 4.2 (Properties of the padded input)**.** The padded input P satisfies:

1. $|P| = n' + 2k + 2ckn'$.
2. $w = w' + 1$ (the dummy box adds one to the width).
3. $k^* = k$ (any monotone classifier must misclassify at least k points in the dummy box; error k is achievable by classifying all non-dummy points correctly).

The key insight is the following:

Lemma 4.2 (Dummy points force correctness)**.** If algorithm A guarantees error $\leq c \cdot k^* = ck$ on the padded input P, then A must correctly classify *every non-dummy point* in P.

Proof. Suppose the classifier h output by A misclassifies some non-dummy point p. Without loss of generality, suppose $\text{label}(p) = -1$ but $h(p) = +1$. By construction, p is dominated by at least ck dummy points that also have label -1. Since h is monotone and maps p to $+1$, it must also map all points dominating p to $+1$, misclassifying at least ck dummy points. Adding p itself, $\text{err}_P(h) \geq ck + 1 > ck^*$, violating the guarantee. □

Since the algorithm must correctly classify all non-dummy points, it effectively solves the realizable problem on the original P'. Therefore, the $\Omega(w' \cdot \text{Log}\, \frac{n'}{w'})$ lower bound from Stage 1 carries over.

4.3 Lower Bound for Arbitrary ε

The previous lower bounds address the exact case and constant approximation ratios. The final result establishes a lower bound for *any* $\varepsilon > 0$, showing that the $1/\varepsilon^2$ dependence in the coreset algorithm is unavoidable.

Theorem 4.3 (Theorem 14 in the paper)**.** Let $0 < \varepsilon \leq 1/10$. Fix integers $n \geq 1$ and $w \geq 1$ such that n is a multiple of w and $n \geq \frac{w \ln n}{\varepsilon^2}$. Any algorithm for Problem 1.1 under $d = 2$ that guarantees expected error $(1 + \varepsilon)k^*$ must probe $\Omega(w/\varepsilon^2)$ elements in expectation on at least one input with width w.

4.3.1 The Hard Input

Fix w distinct locations $x_1, \ldots, x_w \in \mathbb{R}^2$ such that no location dominates another (width w). Place n/w points at each location, each with a distinct ID.

Definition 4.2 (Random labelling)**.** Define:

$$\gamma = 9\varepsilon, \qquad \mu_1 = \frac{1-\gamma}{2}, \qquad \mu_2 = \frac{1+\gamma}{2}.$$

Each location x_i is independently assigned a bias $\mu[i] \in \{\mu_1, \mu_2\}$ uniformly at random. Then each point at location x_i receives label $+1$ with probability $\mu[i]$ and label -1 with probability $1 - \mu[i]$, independently.

Remark 4.4 (Intuition)**.** At each location, the "majority label" is determined by whether $\mu[i] = \mu_2$ (majority $+1$) or $\mu[i] = \mu_1$ (majority -1). The optimal classifier maps x_i to the majority label. The algorithm needs to *infer the majority* at each location, which requires distinguishing μ_1 from μ_2 — two values that differ by only $\gamma = 9\varepsilon$.

4.3.2 The Two Random Processes

The proof introduces two random processes that generate the same distribution over outcomes but differ in *when* labels are assigned.

RP-1 (All labels upfront):

1. Sample $\mu \in \{\mu_1, \mu_2\}^w$ uniformly at random.
2. Label *every* point at x_i with $\mathbf{Pr}[+1] = \mu[i]$.
3. Run algorithm A on the labelled P.

4. Measure $R_1 = \mathrm{err}_P(h_A)/k^*$.

RP-2 (Labels on demand):

1. Sample $\mu \in \{\mu_1, \mu_2\}^w$ uniformly at random.
2. When A probes a point p at x_i, assign its label with $\mathbf{Pr}[+1] = \mu[i]$.
3. After A finishes, label all remaining (un-probed) points.
4. Measure $R_2 = \mathrm{err}_P(h_A)/k^*$.

Lemma 4.3 (Lemma 15 in the paper)**.** $\mathbf{E}[R_1] = \mathbf{E}[R_2]$ and $\mathbf{E}[\Lambda_1] = \mathbf{E}[\Lambda_2]$, where Λ_j is the number of probes in RP-j.

Proof sketch. Both processes assign labels from the same distribution. The only difference is timing: RP-1 assigns all labels before the algorithm runs, RP-2 assigns them as needed. Since the algorithm's decisions depend only on the labels it has *seen* (not unseen ones), the distribution of all observable quantities — including the algorithm's output and cost — is identical. □

Remark 4.5. The purpose of RP-2 is to make the analysis cleaner. In RP-2, we can reason about what the algorithm "knows" after probing: it has seen some labels at each location and must infer $\mu[i]$ from them. This connects to the classical problem of distinguishing two coin biases.

4.3.3 The Distinguishing Barrier

The heart of the lower bound is the following result from statistical learning theory.

Lemma 4.4 (Lemma 17; Anthony–Bartlett)**.** Let μ be a random variable that equals μ_1 or μ_2 each with probability $1/2$. Let $\Sigma = (X_1, \ldots, X_m)$ be a sequence of i.i.d. samples with $\mathbf{Pr}[X_i = 1] = \mu$ and $\mathbf{Pr}[X_i = -1] = 1 - \mu$. If $m < M$ where

$$M = \frac{\ln(9/8) \cdot (1 - \gamma^2)}{\gamma^2},$$

then no algorithm can correctly infer μ from Σ with probability greater than $2/3$.

Remark 4.6. Since $\gamma = 9\varepsilon$, we have $M = \Theta(1/\varepsilon^2)$. This means that distinguishing μ_1 from μ_2 at a single location requires $\Theta(1/\varepsilon^2)$ samples. This is the fundamental bottleneck.

4.3.4 Light and Heavy Locations

We say location i is *light* if $\mathbf{Pr}[A$ probes fewer than M points at $x_i] \geq 3/4$, and *heavy* otherwise.

Lemma 4.5 (Lemma 18 in the paper)**.** If $\mathbf{E}[\Lambda_2] \leq wM/8$, then more than $w/2$ locations are light.

Proof. For each heavy location i, $\mathbf{Pr}[\text{probes at } x_i \geq M] > 1/4$, so $\mathbf{E}[\text{probes at } x_i] > M/4$. If $\geq w/2$ locations were heavy, the total expected probes would exceed $(w/2)(M/4) = wM/8$, contradicting the budget $\mathbf{E}[\Lambda_2] \leq wM/8$. □

Lemma 4.6 (Lemma 19 in the paper)**.** If location i is light, then the algorithm's classifier "guesses wrong" at x_i (i.e., maps x_i to the minority label) with probability $> 1/4$.

Proof. By Lemma 4.4, when fewer than M samples are seen at x_i, the algorithm cannot infer $\mu[i]$ with probability $> 2/3$. It guesses wrong with probability $> 1/3$. Since i is light (fewer than M samples with probability $\geq 3/4$):

$$\mathbf{Pr}[\text{wrong guess}] \;\geq\; \mathbf{Pr}[\text{wrong} \mid \text{few samples}] \cdot \mathbf{Pr}[\text{few samples}] > \frac{1}{3} \cdot \frac{3}{4} = \frac{1}{4}. \qquad \square$$

4.3.5 Completing the Proof

Each wrong guess at location x_i causes the classifier to misclassify the majority at that location, adding excess error of at least $n\gamma/(2w)$. With more than $w/2$ light locations and each contributing excess error with probability $> 1/4$:

$$\mathbf{E}[\text{excess error}] \;\geq\; \frac{w}{2} \cdot \frac{1}{4} \cdot \frac{n\gamma}{2w} = \frac{n\gamma}{16}.$$

The optimal error is $k^* \leq n(1/2 - \gamma/4)$, so the expected approximation ratio satisfies:

$$\mathbf{E}[R_2] \;\geq\; 1 + \frac{n\gamma/16}{n(1/2 - \gamma/4)} > 1 + \frac{153}{144}\varepsilon > 1 + \varepsilon.$$

By Lemma 4.3, $\mathbf{E}[R_1] = \mathbf{E}[R_2] > 1 + \varepsilon$, meaning the algorithm with budget $\leq wM/8 = \Theta(w/\varepsilon^2)$ cannot guarantee expected error $(1 + \varepsilon)k^*$.

4.4 Near-Optimality of the Algorithms

With both upper and lower bounds in hand, we can now compare them side by side. The following table summarizes the results:

Table 3: Matching upper and lower bounds for Problem 1.1

Regime	**Upper Bound**	**Lower Bound**	**Gap**
$\varepsilon = 0$	$O(n)$	$\Omega(n)$ [Thm 4.1]	**Tight**
constant $c > 1$	$O(w \cdot \mathrm{Log}\, \frac{n}{w})$ [RPE]	$\Omega(w \cdot \mathrm{Log}\, \frac{n}{(k^*+1)w})$ [Thm 4.2]	Tight*
any $\varepsilon > 0$	$O\Big(\frac{w}{\varepsilon^2} \mathrm{Log}\, \frac{n}{w} \cdot \log n\Big)$ [Cor 3.1]	$\Omega(w/\varepsilon^2)$ [Thm 4.3]	$O(\mathrm{Log} \cdot \log)$

*Tight when $k^* \leq (n/w)^{1-\delta}$ for any small constant $\delta > 0$.

Several conclusions emerge:

1. $\varepsilon = 0$ **is hopeless.** The $\Omega(n)$ lower bound matches the trivial $O(n)$ upper bound (probe everything). No algorithm can do better.

2. **Width w is the correct complexity parameter.** Both upper and lower bounds scale with w, confirming that the *dominance width* — not n or d — captures the intrinsic difficulty of monotone classification. This is a key structural insight of the paper.

3. **The $1/\varepsilon^2$ dependence is necessary.** The $\Omega(w/\varepsilon^2)$ lower bound shows that the quadratic dependence on $1/\varepsilon$ in the coreset algorithm is not an artifact of the proof technique — it is a fundamental requirement. Halving ε quadruples the cost.

4. **Only a polylogarithmic gap remains.** The gap between the upper bound $O(w/\varepsilon^2 \cdot \mathrm{Log}(n/w) \cdot \log n)$ and the lower bound $\Omega(w/\varepsilon^2)$ is a factor of $O(\mathrm{Log}(n/w) \cdot \log n)$. Closing this gap — from either direction — remains an open problem.

Remark 4.7 (The "true" complexity)**.** The paper concludes that the true complexity of Problem 1.1 is $\Theta(w/\varepsilon^2)$ up to polylogarithmic factors. This is a strong characterization: regardless of the input distribution, dimensionality, or value of k^*, the cost is essentially determined by w and ε alone.

Summary

References

[1] Y. Tao, "Monotone Classification," *PODS'18*.

[2] Y. Tao, "Monotone Classification (Extended)," *PODS'21*.